Universidad Castro Carazo

Portafolio de evidencias

Informática 1

Maria Elena Martinez Garcia

Instrucciones

El portafolio pretende llevar un registro de los aprendizajes de cada semana, generando un documento que evidencia los distintos temas y actividades desarrolladas en clases.

Cada estudiante tiene una visión particular de lo aprendido, por ende, cada portafolio representara la forma que cada uno de interpretar el conocimiento adquirido.

Este documento refleja y será parte del aprendizaje significativo, que usted como estudiante va generando a lo largo del curso, y de las habilidades que con el tiempo ira adquiriendo.

¿Qué vimos?

Esta pregunta responde al conocimiento compartido durante las lecciones, de manera teórica o práctica, por medio de los distintos medios proporcionados por el docente.

¿Qué aprendí?

Cuál fue el aprendizaje significativo, que me quedo de lo visto en clase y las distintas actividades elaboradas, tanto en las lecciones o las actividades asincrónicas.

¿Cómo lo puedo aplicar?

Una vez contestadas las preguntas anteriores, analizo y situó las habilidades y conocimientos adquiridos en mi contexto laboral o de otros indoles en la cual pueda aplicarlos y me sirvan para la mejora de mi desempeño.

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Variables int, input, print	Con estas variables sencilla en python del usuario y den un resultado	Puedo usarlo para crear programas que reciban datos	armar una calculadora calculadoras y otros

Semana 1

Evidencias

```
num1 = int ( input ("Ingrese un numero: "))
num2 = int ( input ("Ingrese un numero: "))



suma = num1 + num2
resta = num1 - num2
multiplicacion = num1 * num2
division = num1 / num2

print ( "suma", suma)
print ("resta:", resta)
print(f"La multplicación de {num1} * {num2} = {multiplicacion}
print(f"La division de {num1} / {num2} = {division}")
```



Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

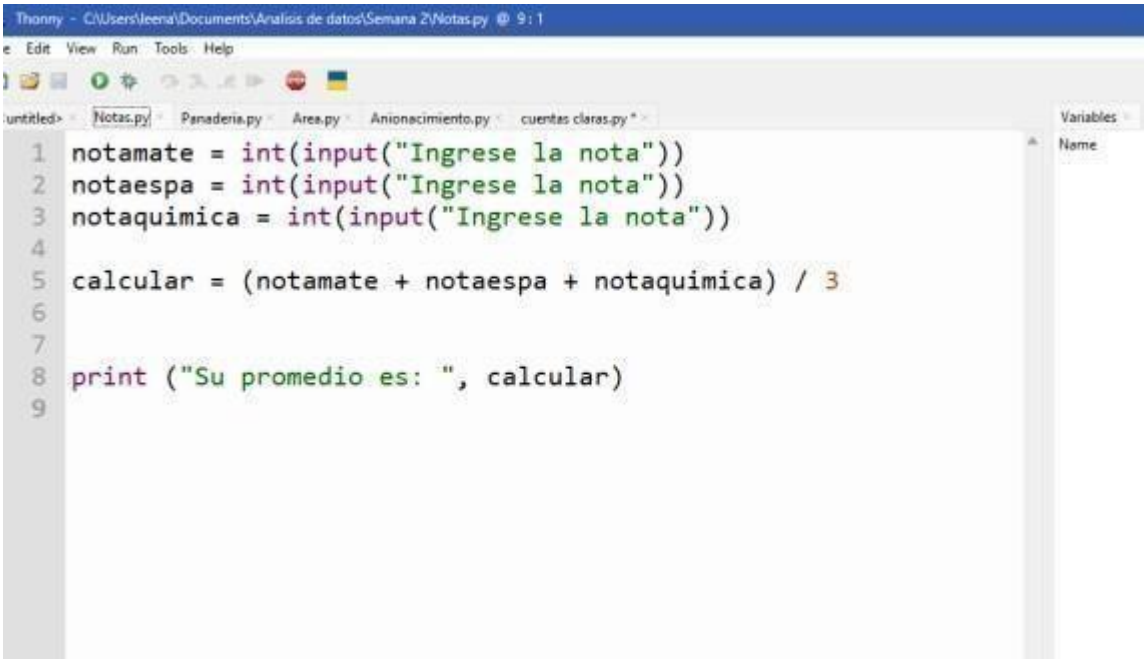
O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

Semana 2

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?

Variable float	Implementar la nueva variable, pedir datos al ingresado Puedo crear calculadoras mas complejas y usuario y realizar varias programas para ecuaciones con los datos diferentes tipos de negocios/necesidades

Evidencias



Thonny - C:\Users\leena\Documents\Analisis de datos\Semana 2\Panaderia.py @ 11 : 33

File Edit View Run Tools Help

<untitled> Notas.py Panaderia.py Area.py Anionacimiento.py cuentas claras.py *

```python
#autor: leena 2026/09/25
#situacion: cada barra de pan cuesta 550 colones, necesitamos
#float es para numeros con decimales
precio = 550 #por unidad
print (type(precio))

cantidad = int(input('Ingrese la cantidad: '))

Total = cantidad * precio

print ("Total a pagar: ", Total)
```

Thonny - C:\Users\leena\Documents\Analisis de datos\Semana 2\Area.py @ 6 : 27

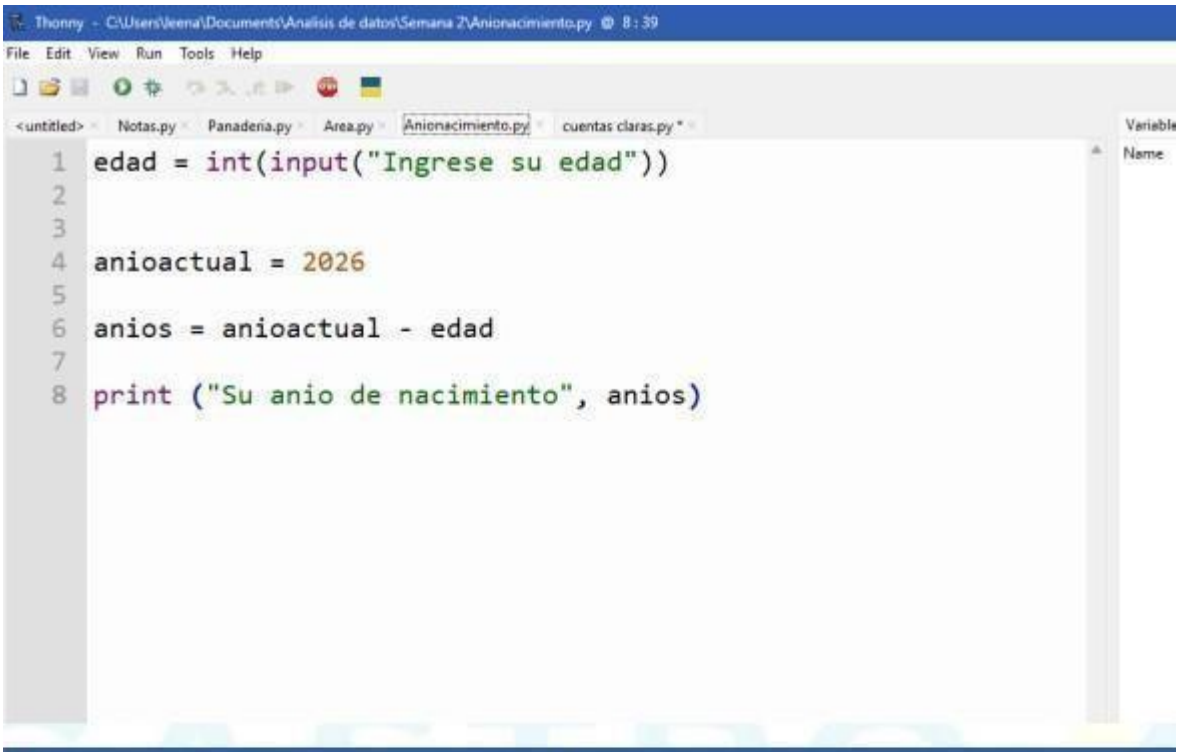
File Edit View Run Tools Help

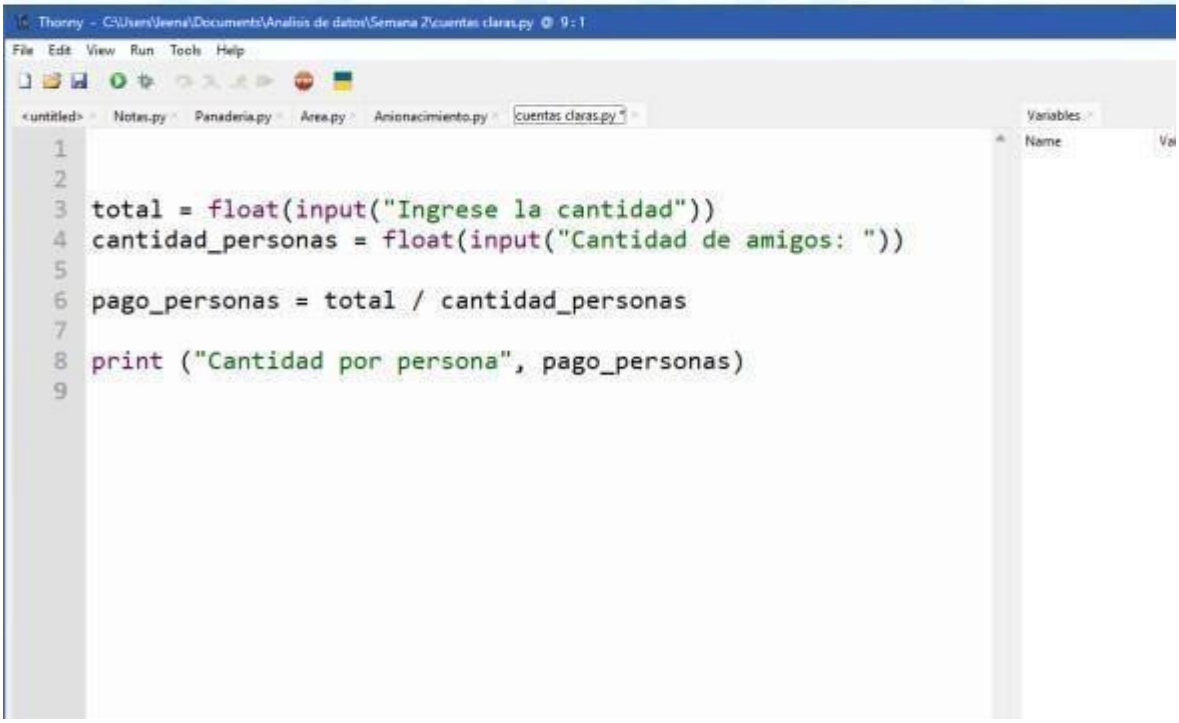
<untitled> Notas.py Panaderia.py Area.py Anionacimiento.py cuentas claras.py *

```python
base = int(input("Ingrese la base"))
altura = int(input("Ingrese la altura"))

area = base * altura / 2

print ("La area es", area)
```





Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

Semana 3

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Variables booleanos	Aprendí que las variables booleanas solo pueden tener dos valores: verdadero o falso. Comprendí que son una condición. Por ejemplo, en	Puedo aplicar las variables booleanas para crear programas que verifiquen si se cumple **true/false, elif, if, else** importantes porque

mi permiten evaluar emprendimiento podría condiciones y ayudan a utilizarlas para que un programa tome comprobar si un producto

decisiones dependiendo está disponible o de si una condición se agotado, y así mostrar un cumple o no. mensaje dependiendo del resultado.

Evidencias

```python
#boleanos true/false

#print (5 == "5")

#numero = 4

#print (numero >= 5)

#usuario = True
#contrasena = True

#print ( "ingreso", usuario and contrasena)
papa = False
mama = False
print("Permiso?: ",papa or mama )
me_escapo = not(mama or papa)

print("Me escapó a bailar: ",me_escapo )


salida = (mama and papa) or me_escapo
```

isa.py | Concierto.py | condiciones_multiples.py | operadores_logicos_y_relacionales.py

```python
#leer una nota. si la nota  es mayor o igual a noventa
#si la nota es mayo a 80 = muy bueno
#si la nota es mayor a 70 = aprobada
# y cualquier nota inferior a 70 es reprobado

nota = float (input("Ingrese una nota (0 a 100)"))

if (nota > 100) or (nota < 0):
    print ("Error")
elif nota >= 90:
    print ("Excelente, felicidades")

elif nota > 80:
    print ("Muy bien")
elif nota >= 70:
    print ("Aprobado")
elif nota < 70 : print ("Reprobado")

```

Visa.py | Concierto.py | condiciones_multiples.py | operadores_logicos_y_relacionales.py

```python
#concierto para mayores de edad

edad = int(input("Ingrese su edad porfavor:"))

if edad >= 18:
    print ("Bienvenido, puedes ingresar")
else:
    print ("No puede ingresar!!!!!!!!!!!!")
```

```python

#Entradas
monto = int(input("Monto en cuenta bancaria ($): "))

antecedentes = input("¿Tiene antecedentes penales? (si/no): ").lower()

profesion = input("Profesión: ").lower()

#Procesos y Salida
if ((monto >= 50000) and (antecedentes == "no")) or (profesion == "diplomatico"):
  print("APROBADO: VISA DORADA ✸")
elif ((monto >= 20000) and (antecedentes == "no")):
  print("APROBADO: VISA ESTÁNDAR ☑")
elif (monto < 20000) and ((profesion == "estudiante") or (profesion == "jubilado")):
  print("PENDIENTE: Requiere entrevista 🖹")
else:
  print("DENEGADO ✖")
```

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

Semana 4

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Vimos cómo usar condiciones **en Python** para que el programa tome datos y den diferentes resultados dependiendo de la información	Aprendí a usar varias condiciones para que el programa revisen **cuando ingresamos datos. Tambien usamos todas las variables** ingresada.	Puedo usarlo para crear programas que revisen decisiones según los datos que recibe. **y cómo evitar errores** **vistas en el curso**

Evidencias

Kits.py

```python
nombre = input("Nombre: ").upper().strip()
kit = input("Tipo de kit: ").lower() .strip()
autorizacion = input("Tiene autorización (si/no)? ").lower() .strip() == 'si'
try:
    cantidad = int(input("Cantidad de Kits?: "))
except ValueError:
    print ("Cantidad de kits invalida, digite del 1 al 100")
    cantidad = -1


try:
  dias = int(input("Días de préstamo: "))

except ValueError:
    print ("Cantidad de dias invalida, digite del 1 al 100")
    dias = -1

if not nombre or not kit or cantidad < 1 or dias < 1:
    resultado = "Registro rechazado, revise los datos"

elif autorizacion and cantidad <= 3 and not dias > 7:
    print (f"Solicitud aprobada para {nombre}: {cantidad} kits de {kit}")

else:
    resultado = "Solicitud enviada a revision"



print ("-" * 50)
print (resultado)
```

Kits.py Microreto_Portero_Cafe.py

```python
"""Microreto: el portero del café."""
nombre = input("Como te llamas?").upper() .strip()
energia = int(input("Cuanta energia tienes del 1-100:"))
traecafe = input("traes cafe? si/no") .lower() .strip() == 'si'

mensaje = "Completa las reglas del portero."

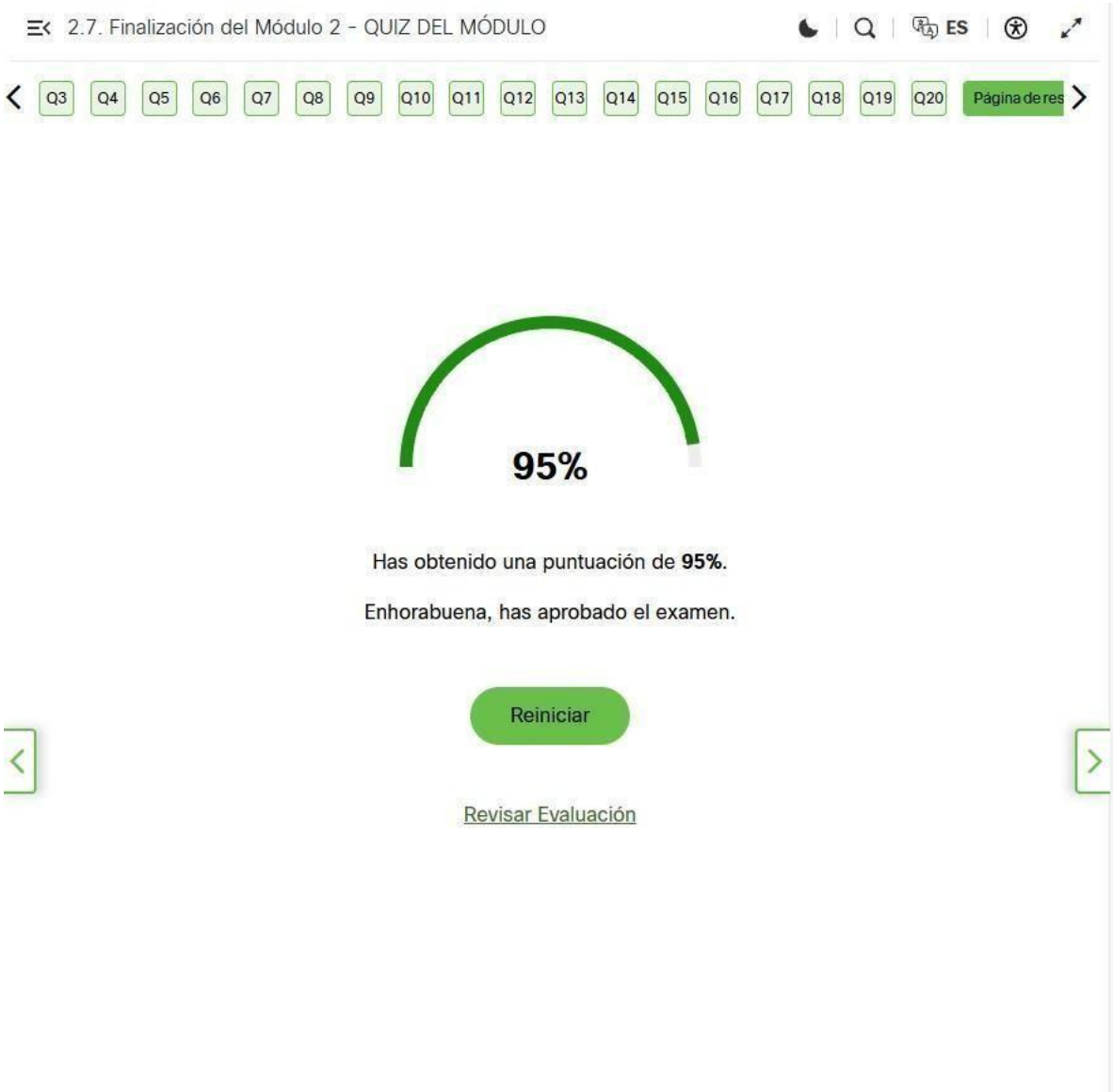
if (energia < 30) and not (traecafe):
    mensaje = "A descansar!!!!!"
    print(mensaje)


elif energia >= 30 or traecafe:
    mensaje = "Acceso permitido"
    print(mensaje)

else:

    mensaje = "El guarda está confundifo, revisa las respuestas."

    print(mensaje)
```



Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

Semana 5

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Listas en Python, operaciones matemáticas y uso de condiciones.	Aprendí a guardar varios datos en lista y realizar cálculos para comparar resultados	Puedo crear programas para calcular ventas, promedios y analizar datos de un negocio.

Evidencias

notaslistas.py

```python
#Una lista es como una caja organizadora
# #            0       1    2    3       <-------- en ese orden se guardan
# milista = ["Elena", 22, 1.64, True] #puede almacenar varios tipos de datos
# print(milista[2]) #asi se imprime un dato en especifico
# print(type(milista))

notas = [85, 60, 98]
sumanotas = sum(notas)
promedio =  sumanotas / 3
if promedio >= 70:
    print ("Felicidades, aprobo!")

elif promedio >= 60:
    print("Va a reposicion")
else:
    print ("Reprobado!!")

print (f"El promedio de notas es: {promedio:.2f}")
```

notaslistas.py | Diagnostico_Semanal_Ventas.py

```python
"""Plantilla guiada: análisis de ventas de dos tiendas, sin ciclos."""

ventas_centro = [125000, 98000, 147000, 132000, 110000]
ventas_norte = [140000, 135000, 150000, 145000, 155000]
meta_centro = 640000
meta_norte = 700000

# TODO: calcula total, cantidad, mayor y menor para Centro.
total_centro = sum(ventas_centro) # Total de ventas semanal
cantidad_centro = len(ventas_centro) #Catidad de días registrados

menor_centro = min(ventas_centro) #Venta minima centro
mayor_centro = max(ventas_centro) #Venta maximo centro
promedio_centro  = total_centro / cantidad_centro

porcentaje_centro = total_centro * 100 / meta_centro

#print(porcentaje_centro)

# TODO: repite los indicadores para Norte.
total_norte = sum(ventas_norte) # Total de ventas semanal
cantidad_norte = len(ventas_norte) #Catidad de días registrados

menor_norte = min(ventas_norte) #Venta minima norte
mayor_norte = max(ventas_norte) #Venta maximo norte
promedio_norte  = total_norte / cantidad_norte

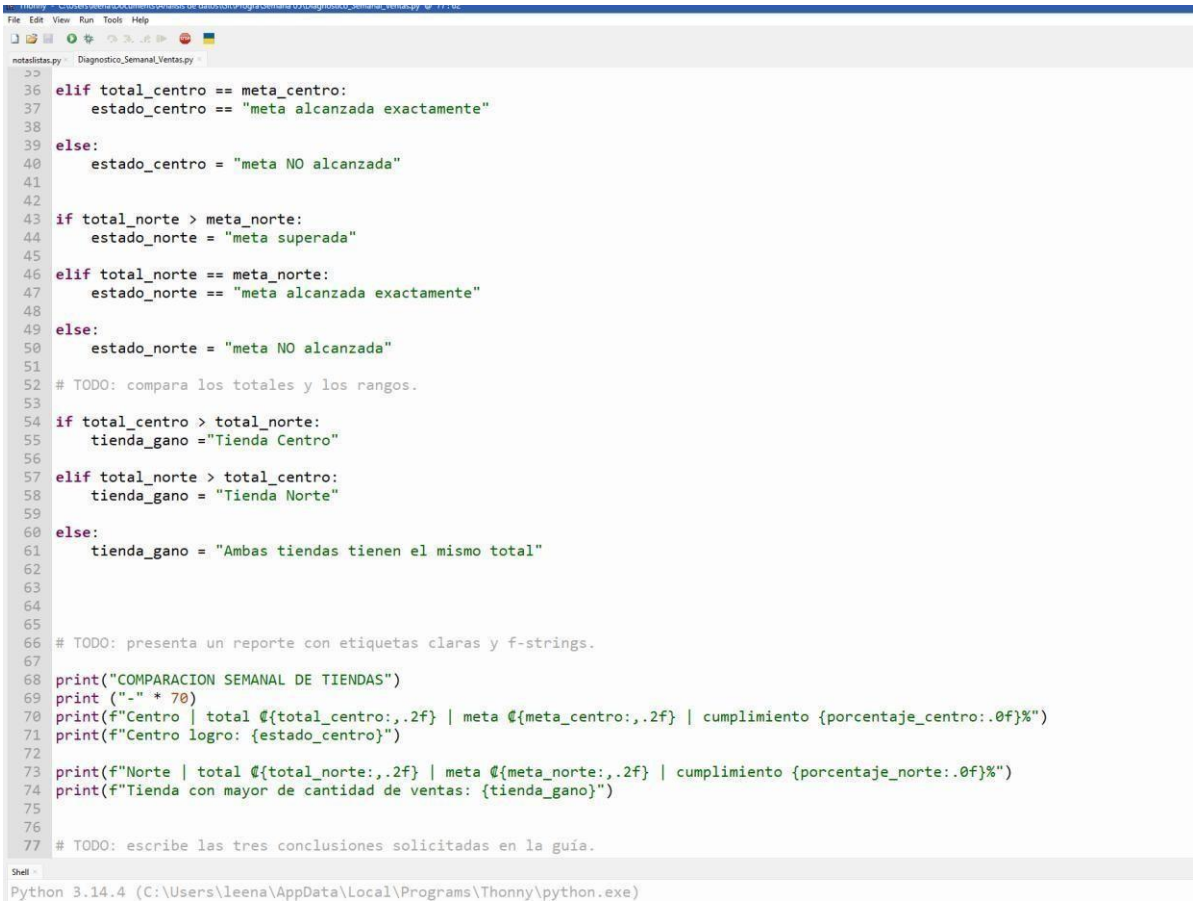
porcentaje_norte = total_norte * 100 / meta_norte

#print(porcentaje_norte)

# TODO: decide si cada tienda superó, igualó o no alcanzó su meta.
if total_centro > meta_centro:
    estado_centro = "meta superada"

elif total_centro == meta_centro:
    estado_centro == "meta alcanzada exactamente"

else:
    estado_centro = "meta NO alcanzada"
```


File Edit View Run Tools Help
notaslistas.py
Diagnostico_Semanal_Ventas.py
elif total_centro == meta_centro:
estado_centro == "meta alcanzada exactamente"
else:
estado_centro = "meta NO alcanzada"
if total_norte > meta_norte:
estado_norte = "meta superada"
elif total_norte == meta_norte:
estado_norte == "meta alcanzada exactamente"
else:
estado_norte = "meta NO alcanzada"
TODO: compara los totales y los rangos.
if total_centro > total_norte:
tienda_gano ="Tienda Centro"
elif total_norte > total_centro:
tienda_gano = "Tienda Norte"
else:
tienda_gano = "Ambas tiendas tienen el mismo total"
TODO: presenta un reporte con etiquetas claras y f-strings.
print("COMPARACION SEMANAL DE TIENDAS")
print ("-" * 70)
print(f"Centro | total ₡{total_centro:,.2f} | meta ₡{meta_centro:,.2f} | cumplimiento {porcentaje_centro:.0f}%")
print(f"Centro logro: {estado_centro}")
print(f"Norte | total ₡{total_norte:,.2f} | meta ₡{meta_norte:,.2f} | cumplimiento {porcentaje_norte:.0f}%")
print(f"Tienda con mayor de cantidad de ventas: {tienda_gano}")
TODO: escribe las tres conclusiones solicitadas en la guía.
Shell
Python 3.14.4 (C:\Users\leena\AppData\Local\Programs\Thonny\python.exe)

notaslistas.py | Diagnostico_Semanal_Ventas.py | Cafe.py

```python
producto = ["Cafe", 5000, 20]
porcentaje_aumento = 20

descuento = producto[1] * 15 /100
precio_promocional = producto[1] - descuento
ingreso_actual = producto[1] * producto[2]
aumento = producto[2] * porcentaje_aumento / 100
producto[2] = producto[2] + aumento
ingreso_proyectado = precio_promocional * producto[2]
diferencia = ingreso_proyectado - ingreso_actual









print("ANÁLISIS DE PROMOCIÓN")
print("-"*50)
print(f"Producto: {producto[0]}", "\n")
print(f"Precio actual: €{producto[1]} ", "\n")
print(f"Unidades actuales: {producto[2] - aumento}", "\n")
print(f"Ingreso actual: €{ingreso_actual}", "\n")
print(f"Descuento: €{descuento}", "\n")
print(f"Precio promocional: €{precio_promocional}", "\n")
print(f"Aumento esperado en ventas: {porcentaje_aumento}%", "\n")
print(f"Unidades proyectadas: {producto[2]}", "\n")
print(f"Ingreso con promoción: {ingreso_proyectado}", "\n")
print(f"Diferencia: {diferencia}", "\n")


if diferencia > 0:
    print ("RECOMENDACION: Lanzar")
else:
    print ("No lanzar")
```

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

Semana 6

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Variables while True .	Aplicar las variables en varios microretos	Lo aplicamos en microretos para entender como usarlos

d

Evidencias

```
"""Micro-reto 2: seleccionar datos con for e if.

Contexto: datos ficticios de horas de estudio semanal reportadas en un cur
universitario de UMCA. Se desea identificar los registros de 3 horas o más

Prediga cuántos registros cumplen la condición antes de completar el ciclo
"""

horas_reportadas = [2, 4, 3, 1, 5]
horas_meta = []

for horas in horas_reportadas:
    if horas >= 3:
        horas_meta.append(horas)
        # TODO: agregue a horas_meta el dato que cumple la condición.
        pass

print("Registros que alcanzan la meta:", horas_meta)
print("Cantidad de registros:", len(horas_meta))

# Resultado esperado:
# Registros que alcanzan la meta: [4, 3, 5]
# Cantidad de registros: 3
```

```python
"""Micro-reto 1: transformar datos con for.

Contexto: datos ficticios de minutos de estudio registrados en un curso
universitario de UMCA.

Prediga el resultado, complete el ciclo y compruebe su respuesta.
"""
#for recorre listas o datos de manera automatica

#                     0   1   2   3
minutos_estudio = [60, 120, 90, 85]
horas_estudio = []

for minutos in minutos_estudio:
    # TODO: convierta los minutos a horas y agregue el resultado a la lista.
    horas = minutos / 60
#     print (horas)
    horas_estudio.append(horas)

print("Horas de estudio:", horas_estudio)

# Resultado esperado: [1.0, 2.0, 1.5]
```

```python
"""Micro-reto 3: actualizar una lista por índice con while.

Contexto: puntajes ficticios de una actividad universitaria expresados como
porcentajes. Para un análisis, se transformarán a valores entre 0 y 1.

Prediga la lista final. Complete la actualización y siga el valor del índice.
"""

puntajes = [72, 85, 90, 100]
indice = 0

while indice < len(puntajes):
    puntajes[indice] = puntajes[indice] / 100
    # TODO: actualice el puntaje de esta posición dividiéndolo entre 100.
    print ("seleccion por indice", puntajes [indice])

    indice += 1

print("Puntajes normalizados:", puntajes)

# Resultado esperado: [0.72, 0.85, 0.9]
```

```
Contexto: se registran resultados ficticios de una actividad universitaria.
La cantidad de registros no se conoce al iniciar.

Ingrese enteros positivos y escriba "fin" para terminar. No ingrese texto
distinto de "fin" ni números negativos en esta práctica.
"""

resultados = []
entrada = input("Resultado (o fin para terminar): ").strip().lower()

while entrada != "fin":
    resultado = int(entrada)
    # TODO: agregue el resultado a la lista.
    resultados.append(entrada)

    entrada = input("Resultado (o fin para terminar): ").strip().lower()

print("Resultados registrados:", resultados)
print("Cantidad de resultados:", len(resultados))

if resultados:
    promedio = sum(resultados) / len(resultados)
    print(f"Promedio: {promedio:.2f}")
else:
    print("No se ingresaron resultados.")

# Con las entradas 80, 65, 90 y fin, se espera:
# Resultados registrados: [80, 65, 90]
# Cantidad de resultados: 3
# Promedio: 78.33
```

for_02_seleccionar_datos.py | for_01_transformar_datos.py | while_01_actualizar_lista.py | while_02_reunir_datos.py | while_03_eliminar_incompletas.py

```
"""Micro-reto 5: eliminar registros incompletos con while y remove().

Contexto: respuestas ficticias de una encuesta universitaria sobre recursos
de estudio. Para este resumen se excluirán las respuestas "Sin respuesta".

Prediga la lista final y cuántas respuestas incompletas se eliminarán.
"""

respuestas = ["Bueno", "Sin respuesta", "Excelente", "Sin respuesta", "Regula
cantidad_incompletas = respuestas.count("Sin respuesta")

while "Sin respuesta" in respuestas:
    # TODO: elimine una respuesta "Sin respuesta" de la lista.
    respuestas.remove("Sin respuesta")

print("Respuestas para analizar:", respuestas)
print("Respuestas incompletas eliminadas:", cantidad_incompletas)

# Resultado esperado:
# Respuestas para analizar: ['Bueno', 'Excelente', 'Regular']
# Respuestas incompletas eliminadas: 2

```

```
# - Texto, 0 o negativos -> entrada invalida y el programa continua.
# """
grupos_aceptados = []
grupos_rechazados = []
capacidad_maxima = 0

entrada = input("Favor ingrese el grupo (o fin para terminar): ").strip().lowe

while entrada != "fin":
    try:
        grupo = int(entrada)
        capacidad_maxima = capacidad_maxima + grupo # 650 + 60 = 710
        if capacidad_maxima <= 700:
            grupos_aceptados.append(grupo)
            print("Grupo aceptado")
        else:
            grupos_rechazados.append(grupo)
            capacidad_maxima = capacidad_maxima - grupo
            print("Grupo rechazado")
        print(f"Ocupación actual: {capacidad_maxima}")
        print(f"Espacios disponibles: {700 - capacidad_maxima} \n")
    except ValueError:
        grupo = -1
    entrada = input("Favor ingrese el grupo (o fin para terminar): ").strip().

personas_admitidas = 0
for grupos in grupos_aceptados:
    personas_admitidas = personas_admitidas + grupos

cantidad_aceptados = len(grupos_aceptados)

print("REPORTE FINAL")
```

for_02_seleccionar_datos.py | for_01_transformar_datos.py | while_01_actualizar_lista.py | while_02_reunir_datos.py | while_03_eliminar_incompletas.py | control_aforo.py

```
113         if capacidad_maxima <= 700:
114             grupos_aceptados.append(grupo)
115             print("Grupo aceptado")
116         else:
117             grupos_rechazados.append(grupo)
118             capacidad_maxima = capacidad_maxima - grupo
119             print("Grupo rechazado")
120         print(f"Ocupación actual: {capacidad_maxima}")
121         print(f"Espacios disponibles: {700 - capacidad_maxima} \n")
122     except ValueError:
123         grupo = -1
124     entrada = input("Favor ingrese el grupo (o fin para terminar): ").strip().
125
126 personas_admitidas = 0
127 for grupos in grupos_aceptados:
128     personas_admitidas = personas_admitidas + grupos
129
130 cantidad_aceptados = len(grupos_aceptados)
131
132 print("REPORTE FINAL")
133 print("-"*80,"\n")
134 print(f"Grupos aceptados: {cantidad_aceptados}")
135 print(grupos_aceptados)
136 print(grupos_rechazados)
137 print(personas_admitidas)
138
139
140
141 # print ("Resultados registrados:", resultados)
142 # print ("Cantidad de resultados registrados:", len(resultados))
143 # Desarrolle su solución a partir de esta línea.
```



Has obtenido una puntuación de **85%**.

Enhorabuena, has aprobado el examen.

Has usado 1 de 2 intentos.

Reiniciar

Evaluación de Revisión

Semana 7

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Nuevas herramientas, diccionarios, tuplas, sets	Aprendí a hacer el reporte de datos más eficiente y rápido con menos líneas de código y mas precisión	Al hacer reportes de datos que son muy extensos

Evidencias

Semana 07 > microretos > 01_lista_ventas.py > ...

```python
"""Microreto 1: recuperar lo aprendido sobre listas.

Prediga la salida, modifique una posición y explique el cambio.
"""

# Una lista conserva el orden: índice 0 = primer día, índice 1 = segundo día.
ventas = [85000, 92000, 78000]
print("Venta del segundo día:", ventas[1])
print("Total inicial:", sum(ventas))

# La venta del segundo día se corrigió de 92000 a 100000.
# TODO: actualice la posición adecuada con una asignación por índice.


ventas[-1] = 100000

print("Segundo día corregido:", ventas[1])
print("Total corregido:", sum(ventas))



# Para explicar: ¿por qué no cambió la cantidad de elementos?
# Para comprobar: el total pasa de 255000 a 263000; aumenta 8000.

```

```python
"""Microreto 2: leer y cambiar un campo de un diccionario.

Un diccionario guarda campos con nombre; cada nombre es una clave.
"""

# A diferencia de ventas[1], aquí se usa una clave como sede["meta"].
# La lista de ventas puede ser uno de los valores del diccionario.
sede = {
    "nombre": "Soda San Pedro",
    "provincia": "San Jose",
    "ventas": [85000, 92000, 78000],
    "meta": 450000,
}

print("Nombre:", sede["nombre"])
print("Primera venta:", sede["ventas"][0])
print("Meta inicial:", sede["meta"])

# La meta registrada cambió a 470000.
# TODO: actualice solo el valor asociado con la clave "meta".

sede ['meta'] = 470000

print("Meta corregida:", sede["meta"])
print("Nombre después del cambio:", sede["nombre"])
print ('Total de ventas:', sum(sede["ventas"]))
# Para explicar: ¿por qué el nombre y las ventas siguen iguales?
# Para comprobar: la meta pasa de 450000 a 470000.

```

```python
"""Microreto 3: crear otro par cuando cambia un resultado.

Una tupla conserva el orden como una lista, pero sus posiciones no se cambian.
"""

# Este par guarda nombre y total semanal; 0 es el nombre y 1 es el total.
par_original = ("Soda San Pedro", 462000)
print("Nombre:", par_original[0])
print("Total original:", par_original[1])

# Otro ejemplo de la misma forma sería ("Panaderia Cartago", 351000).
# Se corrigió el total de Soda San Pedro a 470000.
# TODO: reemplace la línea provisional por una TUPLA NUEVA con nombre y total.
# No use par_original[1] = 470000: eso intenta modificar una tupla.
par_corregido = ('Panaderia Cartago', 351000)


par_original = par_original
print("Total original después:", par_original[1])
print("Total del nuevo par:", par_corregido[1])

# Para explicar: ¿por qué se necesita un par nuevo?
# Para comprobar: el original conserva 462000; el nuevo muestra 470000.

```

Semana 07 > microretos > 04_set_provincias.py > ...

```python
"""Microreto 4: guardar provincias sin repetirlas.

Un set conserva valores únicos. Se agrega con add(), sin posición ni índice.
"""

# También se puede iniciar vacío con set(); {} vacío crea un diccionario.
provincias = {"San Jose", "Cartago"}
print("Cantidad inicial:", len(provincias))
print("¿Está San Jose?:", "San Jose" in provincias)

# TODO: agregue otra vez "San Jose" mediante add().

provincias.add('San Jose')
print("Después de repetir San Jose:", len(provincias))

# TODO: agregue "Heredia" mediante add().
provincias.add('Heredia')
print("Después de agregar Heredia:", len(provincias))
print("¿Está Heredia?:", "Heredia" in provincias)

# Para explicar: ¿por qué repetir San Jose no aumenta la cantidad?
# Para comprobar: las cantidades son 2, 2 y 3; Heredia sí es un dato nuevo.
# Si imprime el set, el orden de las provincias puede variar.

```

Semana 07 > analisis.py > ...

```python
from sedes import sedes

print('Sedes registradas:', len(sedes))
print("Primera sede:", sedes[0]['nombre'])
print('Ventas de la primera sede:',(sedes[0]['ventas']))


print('ANALISIS SEMANAL DE EMPRENDIMIENTOS')

for sede in sedes:
    total=sum(sede['ventas'])
    porcentaje =  total /  sede ['meta'] * 100


    #clasificacion
    if porcentaje >= 100:
        estado = 'Meta alcanzada'
    elif porcentaje >= 80:
        estado = 'Cerca de la meta'
    else:
        estado = 'Requiere atencion'

    print(
        f"{sede['nombre']} Total: {total:,.2f}"
        f"Cumplimiento: {porcentaje:.2f}% : {estado}"

    )

```

```python
"""Diez sedes ficticias EmprendeCR para analizar en un archivo nuevo.

Las primeras seis proceden del caso anterior; las últimas cuatro amplían el
escenario de práctica. Mantenga los registros sin cambios durante la actividad.
"""

sedes = [
    {
        "nombre": "Soda San Pedro",
        "provincia": "San Jose",
        "tipo": "Alimentacion",
        "ventas": [85000, 92000, 78000, 110000, 97000],
        "meta": 450000,
    },
    {
        "nombre": "Panaderia Cartago",
        "provincia": "Cartago",
        "tipo": "Panaderia",
        "ventas": [65000, 70000, 72000, 69000, 75000],
        "meta": 380000,
    },
    {
        "nombre": "Cafeteria Liberia",
        "provincia": "Guanacaste",
        "tipo": "Cafeteria",
        "ventas": [98000, 105000, 99000, 120000, 115000],
        "meta": 500000,
    },
    {
        "nombre": "Feria de Heredia",
        "provincia": "Heredia",
        "tipo": "Feria",
        "ventas": [72000, 76000, 80000, 78000, 74000],
        "meta": 390000,
    },
    {
        "nombre": "Marisqueria Puntarenas",
        "provincia": "Puntarenas",
        "tipo": "Restaurante",
        "ventas": [115000, 108000, 112000, 118000, 121000],
        "meta": 560000,
    },
    {
        "nombre": "Tienda Escazu Local",
        "provincia": "San Jose",
        "tipo": "Comercio",
        "ventas": [88000, 91000, 87000, 94000, 90000],
        "meta": 460000,
    },
    {
        "nombre": "Libreria Alajuela",
        "provincia": "Alajuela",
        "tipo": "Comercio",
        "ventas": [90000, 95000, 100000, 85000, 105000],
        "meta": 480000,
    },
    {
        "nombre": "Tienda Limon Centro",
        "provincia": "Limon",
        "tipo": "Comercio",
        "ventas": [70000, 80000, 75000, 85000, 90000],
        "meta": 390000,
    },
    {
        "nombre": "Ferreteria Desamparados",
        "provincia": "San Jose",
        "tipo": "Ferreteria",
```

Activate Windows
Go to Settings to activate Windows.

```
        "provincia": "Puntarenas",
        "tipo": "Restaurante",
        "ventas": [115000, 108000, 112000, 118000, 121000],
        "meta": 560000,
    },
    {
        "nombre": "Tienda Escazu Local",
        "provincia": "San Jose",
        "tipo": "Comercio",
        "ventas": [88000, 91000, 87000, 94000, 90000],
        "meta": 460000,
    },
    {
        "nombre": "Libreria Alajuela",
        "provincia": "Alajuela",
        "tipo": "Comercio",
        "ventas": [90000, 95000, 100000, 85000, 105000],
        "meta": 480000,
    },
    {
        "nombre": "Tienda Limon Centro",
        "provincia": "Limon",
        "tipo": "Comercio",
        "ventas": [70000, 80000, 75000, 85000, 90000],
        "meta": 390000,
    },
    {
        "nombre": "Ferreteria Desamparados",
        "provincia": "San Jose",
        "tipo": "Ferreteria",
        "ventas": [50000, 60000, 55000, 65000, 70000],
        "meta": 400000,
    },
    {
        "nombre": "Vivero Paraiso",
        "provincia": "Cartago",
        "tipo": "Vivero",
        "ventas": [65000, 70000, 60000, 75000, 80000],
        "meta": 350000,
    },
```

Semana 8

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Trabajamos con VS Studiom, usar los comandos from e import, el uso de url_datos para usar datos de un URL	A usar datos que están siendo actualizados constantemente	Puedo crear sistemas que se actualicen y muestren datos sin tener que manualmente ingresarlos

Evidencias

conexion_api.py × analisis_en_vivo.py

Semana 08 > semana08_material_estudiante > conexion_api.py > cargar_datos

```python
"""Lectura sencilla de los registros públicos de la Semana 8."""

import json
from urllib.error import URLError
from urllib.request import Request, urlopen

# Esta dirección apunta al JSON de atenciones publicado en GitHub.
URL_DATOS = (
    "https://api.github.com/repos/amenaactppp/td10-datos-publicos/contents/datos/registros.json?ref=ma
)


def cargar_datos():
    """Lee el JSON público y devuelve una lista de atenciones."""
    # GitHub recibe la dirección y estas cabeceras para entregar el JSON.
    solicitud = Request(URL_DATOS, headers={
        "Accept": "application/vnd.github.raw+json",
        "User-Agent": "TD10-Semana08-Clase",
    })

    try:
        # Abrimos la conexión y convertimos el JSON en datos de Python.
        with urlopen(solicitud, timeout=10) as respuesta:
            registros = json.load(respuesta)

        # El análisis necesita una lista: cada elemento es una atención.
        if not isinstance(registros, list):
            raise TypeError("La fuente no contiene una lista de atenciones")
    except (URLError, TimeoutError, json.JSONDecodeError, TypeError, OSError) as error:
        raise SystemExit(f"No se pudieron leer los datos públicos: {error}") from error

    return registros

```

Semana 08 > semana08_material_estudiante > analisis_en_vivo.py > ...

```python
"""Semana 8: informe de atenciones hospitalarias ficticias.

Complete las fases con el docente. La lectura de datos ya está preparada.
"""

"""Semana 8: informe de atenciones hospitalarias ficticias.

Complete las fases con el docente. La lectura de datos ya está preparada.
"""

# FASE 0: CONSUMIR LA CONEXIÓN PREPARADA
# conexion_api.py se encuentra junto a este archivo.
from conexion_api import cargar_datos

registros = cargar_datos()
print("Fuente: JSON público de GitHub")
print("Atenciones recibidas:", len(registros))

# La muestra solicitada usa range(3): deben existir las posiciones 0, 1 y 2.
if len(registros) < 3:
    raise SystemExit("Se necesitan al menos 3 registros. Publique la etapa 3.")

# FASE 1: MOSTRAR Y CONTAR LOS MEDICAMENTOS DE TRES ATENCIONES
# Cada clave es un nombre de medicamento y su valor es el nombre estandarizado
equivalencias = {
    "acetaminofen" : "acetaminofén",
    "paracetamol": "acetaminofén",
    "antiacido": "antíacido",
    "lorsatán": "lorsatan"
}
conteo_medicamentos = {}
total_medicamentos = 0
print("\n\tANALISIS FASE #1 MUESTRA 3")
for indice in range (3):
    registro = registros[indice]
    print(f"\nCédula: {registro['paciente_cedula']}"
        f"\n- Paciente: {registro['paciente_nombre']}"
        f"\n- Diagnóstico: {registro['diagnostico']}"
        f"\n- Medicamentos: {registro['medicamentos']}")

    for medicamento in registro['medicamentos']:
        #Limpieza PARACETAMOL -> paracetamol -> aceteminofén
        medicamento = medicamento.strip().lower()
        if medicamento in equivalencias:
            medicamento = equivalencias[medicamento]
        #Contar medicamentos
        total_medicamentos += 1
        if medicamento in conteo_medicamentos:
```

```
48            if medicamento in conteo_medicamentos:
49                conteo_medicamentos[medicamento] += 1
50            else:
51                conteo_medicamentos[medicamento] = 1
52
53    print("\nTotal de medicamentos recetados:", total_medicamentos)
54    print(conteo_medicamentos)
55
56    # FASE 2: CONTAR LOS MEDICAMENTOS DE TODAS LAS ATENCIONES
57    print("\nANALISIS FASE #2 MEDICAMENTOS Y ATENCIONES")
58    conteo_medicamentos = {}
59    total_medicamentos = 0
60    for registro in registros:
61
62        for medicamento in registro['medicamentos']:
63
64            #Limpieza PARACETAMOL -> paracetamol -> aceteminofén
65            medicamento = medicamento.strip().lower()
66            if medicamento in equivalencias:
67                medicamento = equivalencias[medicamento]
68            #Contar medicamentos
69            total_medicamentos += 1
70            if medicamento in conteo_medicamentos:
71                conteo_medicamentos[medicamento] += 1
72            else:
73                conteo_medicamentos[medicamento] = 1
74
75    print("\nTotal de medicamentos recetados:", total_medicamentos)
76    for medicamento, cantidad in conteo_medicamentos.items():
77        print(f"- {medicamento}: \t{cantidad}")
78
79    # FASE 3: CONTAR ATENCIONES POR HOSPITAL
80    print("\nANALISIS FASE #3 HOSPITALES Y ATENCIONES")
81    conteo_hospital = {}
82    total_pacientes = 0
83    for registro in registros:
84        total_pacientes += 1
85        if registro['hospital'] in conteo_hospital:
86            conteo_hospital[registro['hospital']] += 1
87        else:
88            conteo_hospital[registro['hospital']] = 1
89    print("\nTotal de pacientes atendidos:", total_pacientes)
90    for hospital, cantidad in conteo_hospital.items():
91        print(f"- {hospital}: \t{cantidad}")
92
93    # FASE 4: CONTAR ATENCIONES POR ENFERMEDAD DIAGNOSTICADA
94    print("\nANALISIS FASE #4 ENFERMEDADES Y ATENCIONES")
95    conteo_diagnostico = {}
96    total_diagnosticos = 0
```

Problems Output Debug Console Terminal Ports Python: analisis_en_vivo

Semana 08 > semana08_material_estudiante > analisis_en_vivo.py > ...

```python
        else:
            conteo_medicamentos[medicamento] = 1

print("\nTotal de medicamentos recetados:", total_medicamentos)
for medicamento, cantidad in conteo_medicamentos.items():
    print(f"- {medicamento}: \t{cantidad}")

# FASE 3: CONTAR ATENCIONES POR HOSPITAL
print("\nANALISIS FASE #3 HOSPITALES Y ATENCIONES")
conteo_hospital = {}
total_pacientes = 0
for registro in registros:
    total_pacientes += 1
    if registro['hospital'] in conteo_hospital:
        conteo_hospital[registro['hospital']] += 1
    else:
        conteo_hospital[registro['hospital']] = 1
print("\nTotal de pacientes atendidos:", total_pacientes)
for hospital, cantidad in conteo_hospital.items():
    print(f"- {hospital}: \t{cantidad}")

# FASE 4: CONTAR ATENCIONES POR ENFERMEDAD DIAGNOSTICADA
print("\nANALISIS FASE #4 ENFERMEDADES Y ATENCIONES")
conteo_diagnostico = {}
total_diagnosticos = 0
for registro in registros:
    total_diagnosticos += 1
    if registro['diagnostico'] in conteo_diagnostico:
        conteo_diagnostico[registro['diagnostico']] += 1
    else:
        conteo_diagnostico[registro['diagnostico']] = 1
print("\nTotal de diagnósticos realizados:", total_diagnosticos)
for diagnostico, cantidad in conteo_diagnostico.items():
    print(f"- {diagnostico}: \t{cantidad}")
```

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.